Introduction to RxJava

Student Sheet

Introduction

In this exercise sheet, we will have a first contact with RxJava, its observables, and event stream processing. In particular, we will practice with:

- Creating observables
- The difference between cold and hot observables
- Transformation and filtering operators

1 Exercise Statements

Module 1. Observable Types and Creation

Exercise 1. A basic stream

Create an `Observable` that emits the names of the first five planets in the solar system and then completes.

Exercise 2. Difference between "just" and "from"

Create one observable with `Observable.just()` from a list and another one with `Observable.fromIterable()`. Observe the difference in how list elements are emitted.

Exercise 3. Equivalence between creation methods

Create equivalent observables using `just`, `fromArray`, `range`, and `create` that emit integers from 1 to 3. Check that they produce the same output.

Exercise 4. Using `Single`

Create a `Single` that loads the content of file `file.txt` as a `Stream<String>` and handles both success and error cases.

Exercise 5. An optional result with `Maybe`

Create a `Maybe` that emits "Ok" only if a random number between 0 and 1 is greater than 0.5; otherwise, throw an exception.

Exercise 6. A completable task

Use a `Completable` to simulate a disk write operation that takes 2 seconds and then completes.

Exercise 7. Periodic event emission

Use `Observable.interval()` to emit one event every 500 ms.

Exercise 8. Lazy execution

Create an observable that prints "Generating a value" inside the emitter before emitting a random number between 0 and 1. Subscribe twice to this observable. How many times is the message printed?

Exercise 9. Subscription cancellation

Create an observable that emits one number every second (`interval`). Cancel the subscription after 3 emissions.

Module 2. *Cold* and *Hot* Observables

Exercise 10. Cold start

Create a "Cold" observable that emits a random number. Subscribe two observers with a delay between them and note that each one gets different numbers.

Exercise 11. From cold to hot

Create a "Cold" observable with `interval()` to emit every second. Convert it to "Hot" using `publish()` and then invoke `connect()` to start emission. Observe that both observers receive the same numbers. However, a subscriber only receives emissions produced after subscription. Try invoking `.connect()` after one or both subscriptions.

Exercise 12. On time or late arrival

Create an observable that emits every second (`interval()`). Convert it to "Hot" with `publish().refCount()`. Subscribe two observers with a 3-second delay and observe the sequence each one receives.

Exercise 13. Subject as both Observable and Observer

Create a `PublishSubject`. Manually invoke `onNext()` on this object and show that each subscriber only sees events that happen after subscribing.

Module 3: Operators (Transform & Filter)

Exercise 14. From number to text

From an integer sequence (1 to 10), use `map()` to produce the square of each number as text, for example: "1 squared is 1".

Exercise 15. Temperature converter

Given a sequence of temperatures in Celsius, use `map()` to convert them to Fahrenheit.

Exercise 16. Flattening the result

Consider method `getOrderIds()` that simulates obtaining order identifiers for a given customer. Starting from an observable that emits customer identifiers, produce the sequence of their order identifiers. Try `map()` and `flatMap()` to see the difference.

Exercise 17. Filtering

Create a sequence of 100 random integers and filter only those that are prime. You can use method `isPrime()`.

Exercise 18. Unique values

Use `distinct()` on a sequence with repeated values to ensure that only unique values are emitted.

Exercise 19. Take While

Use `takeWhile()` on an observable that emits integers to stop as soon as the number is greater than 50.